OFFICIAL TECHNICAL DOSSIER • PRODUCTION & SECURITY RELEASE

Humanity Ledger

Comprehensive Engineering, Architecture & Cryptographic Security Overhaul

Target Audience: Aztec Network Community & Protocol Auditors

Verification: 671 / 671 Unit Tests Passing • 0 TypeScript Errors

Date: October 2026

Executive Summary

This dossier provides a comprehensive, transparent, and authoritative review of the systematic transformation executed across the **Humanity Ledger** codebase. Over recent engineering cycles, the project has transitioned from demonstrative prototypes to an institutional, non-custodial cryptographic platform built with zero simulation tolerance.

A total of **17 distinct security vulnerabilities** across authentication, session management, perimeter routing, enclave PIN protection, and internal cron schedulers were diagnosed and resolved. Concurrently, the platform has integrated **Ledger Studio** (autonomous agent code engineering), modernized its **Aztec Network v5.0.1 integration**, hardened decentralized **XMTP MLS messaging**, and decommissioned more than **200 dead or insecure legacy API endpoints**.

TypeScript Strict Typecheck

0 Errors

Validated via tsc --noEmit across entire workspace

Vitest Unit Test Suite

671 / 671 Passing

34 test suites covering cryptography, auth, & resilience

> **Core Architectural Pledge:** Zero custodial control over user funds, zero server-side storage of plaintext communication, zero artificial data synthesis, and mathematically provable client-side Zero-Knowledge proving.

1. Paradigm Shift: From Simulation to Cryptographic Rigor

Prior iterations of the system contained synthetic fallback routines designed for demonstration purposes. These have been eradicated in favor of honest, verifiable system behavior:

- **Elimination of Pseudo-Random Sequencer Data:** In `app/api/zk/sequencer/route.ts`, all pseudo-random simulation loops using `Math.random()` to fabricate block production were removed. If an active Aztec Sequencer RPC node is unavailable, the endpoint truthfully reports a `disabled` or `unreachable` status rather than generating synthetic batches.
- **Decommissioning of Simulated Gasless Paymasters:** Legacy routes such as `/api/premium/paymaster` and `/api/premium/webhooks` that returned hardcoded sponsor signatures (e.g. `0xLedgerNetworkGasSponsorSignature...`) without executing on-chain sponsorship were retired (HTTP 410) and excised from all client flows.
- **Strict Non-Custodial Cryptographic Model:** The server neither derives, stores, nor possesses Ethereum private keys, Aztec viewing keys, or XMTP installation credentials. All Zero-Knowledge proof generation occurs strictly client-side.

2. Aztec Network Integration & Client-Side ZK Architecture

Humanity Ledger aligns with Aztec Network standards to provide programmable privacy and non-custodial asset management:

- **Client-Side PXE Proving:** Proof computation is offloaded to `public/workers/prover.worker.js` in isolated Web Worker threads. Pages serving proof generation are configured with Cross-Origin Opener Policy (`same-origin`) and Cross-Origin Embedder Policy (`require-corp`) to enable high-performance multi-threaded WebAssembly execution.
- **Noir Smart Contract Invariants:**
  - `zk_escrow` : Conditional escrow releases governed by authorized witness (authwit) cryptographic signatures.
  - `zk_streams` : Continuous payment streaming contracts with block-timestamp verification and non-repudiation.
- **On-Chain Provenance Anchoring Integrity:** In `app/api/passport/[slug]/anchor/route.ts` , anchoring proofs submitted to the ledger are strictly validated against a 32-byte hexadecimal format (`/^0x[0-9a-fA-F]{64}$/`). The endpoint verifies issuer ownership and enforces idempotency to prevent transaction hash overwrites.
- **Resilient RPC Circuit Breaking:** Network calls to external RPC providers implement automated circuit breakers with exponential backoff, preventing application cascade failures during sequencer latency spikes.

3. Ledger Studio: Autonomous Code Engineering Agent

Ledger Studio provides an integrated development workspace facilitating agentic code modification and testing under strong isolation boundaries:

- **Agent Tooling Protocol:** Built on autonomous agent primitives inspired by Claude Code. The agent employs exact string target replacement for edits, granular file reading by byte ranges, directory exploration, and automated sandbox command execution.
- **Workspace Virtualization:** Supports branch creation and isolated directory switching, permitting speculative code modifications, test runs, and static scans without modifying the active working tree.
- **GitHub Synchronization:** Bidirectional integration enabling branch pulling, diff generation, and code review directly within the sovereign desktop interface.
- **Execution Sandboxing:** Development terminal commands run in a restricted sandbox by default, disallowing arbitrary external network egress and unmonitored file access outside the project tree unless explicitly approved by the operator.
- **Per-Turn Metering & Safeguards:** Implements deterministic execution locks, tool step retries, and strict per-turn execution bounds to prevent infinite tool loops.

4. Cryptographic Messaging, Privacy & Community Sandbox

Communication layers are architected to guarantee privacy, authenticity, and resilience against surveillance:

- **XMTP MLS End-to-End Encryption:** Direct messages travel across the decentralized XMTP network using MLS (Messaging Layer Security) with dynamic ratcheting. The central PostgreSQL database retains zero message contents.
- **Offline Message Outbox Hardening:** When recipients are offline, temporary payloads in `PendingChatMessage` are stored strictly encrypted. Messages are enforced with a 7-day automatic time-to-live expiration, a 10MB payload ceiling, and rate-limiting at 90 messages per minute per account.
- **Encrypted Voice/Video Calling with IP Concealment:** Audio and video calling powered by LiveKit uses frame-level encryption with keys rotated upon participant join/leave. When enabled, `iceTransportPolicy: 'relay'` routes traffic exclusively through TURN relays, completely masking participant IP addresses.
- **Community Application Sandboxing:** User-generated community applications run inside sandboxed iframes governed by `sandbox allow-scripts` with a Content-Security-Policy that denies all network egress (`connect-src 'none'`) and forbids storage access. Interaction with the parent application is restricted to an authenticated, parent-moderated `postMessage` bridge.

5. Comprehensive Security Audit & Vulnerability Remediation Registry

The following table summarizes all identified vulnerabilities, their severity, root causes, and the concrete mitigaciones implemented and verified in the codebase:

Code	Target / Endpoint	Severity	Vulnerability Description & Engineering Fix
A-018	`/api/premium/webauthn`	CRITICAL	**Session Forging / Account Takeover:** Route signed valid sessions using an unverified cookie value as subject. Route decommissioned (HTTP 410); webauthn-signed tokens revoked at Edge.
A-019	`middleware.ts`	HIGH	**Matcher Regex Bypass:** Lookahead regex skipped middleware for paths containing extensions like `.png`, bypassing CSRF, session verification, and route retirement. Corrected with anchored static asset checks.
A-020	`/api/graph`	HIGH	**Arbitrary Cypher Injection:** POST handler allowed authenticated callers to execute unparameterized Cypher queries against Neo4j. Route retired and blocked at Edge (HTTP 410).
A-021	`/api/auth/enclave-pin`	HIGH	**PIN Brute-Force & Race Conditions:** DELETE lacked attempt tracking; POST/PUT had parallel race windows. Replaced with shared limiter counting attempts *before* cryptographic verification (max 8 per 5 min).
A-022	`GeoLoginTracker`	MEDIUM	**Unencrypted IP / Location Leak:** Transmitted visitor IP and coordinates over plaintext HTTP to `ip-api.com`. Tracker deleted; `/api/geo/login-event` retired.
A-023	`/api/cron/*` daemons	HIGH	**Fail-Open Schedulers:** Allowed public execution when `CRON_SECRET` was unset in development/production. Unified under `cronAuth.ts`; strictly fails closed if secret is missing or under 16 characters.
A-024	`/api/auth/qr-sync-seed`	HIGH	**Seed Exposure on Null Creator:** If seed was pushed without session, `creatorAddress` was null and GET allowed any authenticated caller to read the ciphertext. Enforced strict 403 rejection if creator is null.
A-025	`/api/pulse` / daemons	HIGH	**Runtime Random Secret Cold-Start:** Code fell back to `crypto.randomBytes(32)` on each cold start, invalidating authorized scheduler calls. Migrated to `cronAuth.ts` fail-closed validation.
A-026	`/api/variables`	MEDIUM	**IDOR via Query Fallback:** Fell back to `searchParams.get('wallet')` when Edge session header was absent. Fallback removed; requires verified Edge header exclusively.
A-027	`/api/user/search`	MEDIUM	**Unauthenticated User Enumeration:** Publicly exposed user wallet addresses, bios, and ZK verification statuses. Enforced mandatory verified caller session before executing database queries.
A-028	`/api/notifications/ingest`	HIGH	**Timing Attack & Random Key:** Non-constant-time key check and random secret fallback. Fixed with SHA-256 `timingSafeEqual` comparison and fail-closed secret validation.

Code	Target / Endpoint	Severity	Vulnerability Description & Engineering Fix
A-029	`/api/vault/sync` / audit	MEDIUM	**Timing-Attack Leak in Vault Sync:** String equality checks on authorization headers allowed side-channel latency profiling. Upgraded to constant-time comparison on cryptographic digests.
A-030	`/api/bots/command`	LOW	**Quota Exhaustion & Path Traversal:** Unrestricted third-party API queries. Implemented IP rate-limiting (30 req/min) and strict regex validation forbidding path traversal in ENS lookups.
A-031	`/api/academy/parse-document`	MEDIUM	**Zip Bomb Denial of Service:** Extraction of docx files trusted archive headers. Hardened with 10MB upload ceiling and 25MB maximum decompressed buffer cap using `inflateRawSync`.
A-032	`/login` & `/connect`	MEDIUM	**Open Redirect / DOM XSS:** Post-login `?next=` parameter accepted external URLs or javascript schemes. Unified under `safeRedirect.ts` validating relative local paths only.
A-033	`/api/admin/clean-posts`	MEDIUM	**CSRF in Administrative Deletion:** Database deletion ran on HTTP GET, vulnerable to cross-site image navigation. Migrated to POST under Fetch Metadata CSRF enforcement.
A-034	Edge Routing Registry	LOW	**Extensive Attack Surface:** Over 200 unused, unauthenticated experimental endpoints were exposed. Cataloged in `retiredRoutes.ts` and answered with HTTP 410 Gone at the Edge.

6. Verification Suite, Testing & Code Quality Assurance

Every architectural and security modification has been validated against strict automated quality gates:

- **TypeScript Strict Compilation:** Completed with **0 errors** via `tsc --noEmit`. No type bypasses (`any`) were introduced into cryptographic verification paths.
- **Unit & Integration Test Suite:** All **671 tests passed** across 34 suites in Vitest. Specific security suites cover:
  - `audit-2026-10-29.test.ts` : Verified Edge matcher, cron auth fail-closed behavior, QR seed ownership, variables IDOR prevention, and user search gating.
  - `audit-2026-10-28c.test.ts` : Verified enclave PIN rate-limiting across concurrent DELETE, POST, and PUT operations.
  - `audit-2026-10-28b.test.ts` : Verified CSP nonce hydration, open redirect defenses, and session claim validation.
  - `circuit-breaker.test.ts` : Verified circuit state transitions (CLOSED → OPEN → HALF-OPEN).
  - `eip191-verify.test.ts` : Verified Ethereum cryptographic signature validation.
- **Zero Security Regression:** Clean git status on branch `claude/sync-and-hardening` with clean repository hygiene.

7. Production Deployment & Operational Security Protocol

For system administrators deploying Humanity Ledger into live environments, the following procedures are mandatory:

1. **Immediate Cryptographic Key Rotation:** Rotate `JWT_SECRET` and the EdDSA asymmetric keypair (`JWT_EDDSA_PRIVATE_JWK` / `JWT_EDDSA_PUBLIC_JWK`) in deployment environment variables to revoke all legacy tokens and sessions.
2. **Mandatory High-Entropy Variables (Minimum 16 Characters):**

   ```
   CRON_SECRET=your_32_byte_secure_hex_string_here
   INTERNAL_INGEST_KEY=your_32_byte_secure_hex_string_here
   Private_VAULT_SECRET=your_32_byte_secure_hex_string_here
   ADMIN_SECRET=your_32_byte_secure_hex_string_here
   PULSE_SECRET=your_32_byte_secure_hex_string_here
   ENCLAVE_PIN_PEPPER=your_32_byte_secure_hex_string_here
   ```

   Note: Under the hardened architecture, missing or short secrets cause corresponding endpoints to fail closed (HTTP 503/401).
3. **Dedicated TURN Infrastructure:** In enterprise production environments, configure dedicated Coturn servers via `NEXT_PUBLIC_TURN_URLS` , `NEXT_PUBLIC_TURN_USERNAME` , and `NEXT_PUBLIC_TURN_CREDENTIAL` to eliminate reliance on public community fallback relays.
4. **Aztec Node Connectivity:** Ensure `AZTEC_NODE_URL` points to a healthy Aztec Network v5.0.1 node to enable real-time sequencer status reporting and client-side proof verification.